```
==========================================================
==========
DEVOPS LAB (U21PC683IT) - ANSWERS
MVSR Engineering College (A) | Dept. of IT | B.E VI
Sem 2026
==========================================================
==========
==========================================================
==========
COMMON QUESTIONS (Asked in EVERY Program Set)
==========================================================
==========
----------------------------------------------------------
------
Q1. List out any five Git commands and give its usage.
----------------------------------------------------------
------
1. git init
   - Initializes a new Git repository in the current
folder.
   - Example: git init
2. git clone <url>
   - Copies a remote repository to your local machine.
   - Example: git clone
https://github.com/user/repo.git
3. git add .
   - Stages all changed files to be included in the
next commit.
   - Example: git add .
4. git commit -m "message"
   - Saves the staged changes with a description.
   - Example: git commit -m "first commit"
5. git push origin main
   - Uploads your local commits to the remote
repository (GitHub).
   - Example: git push origin main
----------------------------------------------------------
------
Q2. List out any five Docker commands and give its
usage.
----------------------------------------------------------
------
1. docker pull <image>
   - Downloads a Docker image from Docker Hub.
   - Example: docker pull ubuntu
2. docker build -t myapp .
   - Builds a Docker image from a Dockerfile in the
current folder.
   - Example: docker build -t myapp .
3. docker run <image>
   - Creates and starts a container from an image.
   - Example: docker run hello-world
4. docker ps
   - Lists all currently running containers.
   - Example: docker ps
5. docker push <username/image>
   - Pushes a local Docker image to Docker Hub.
   - Example: docker push john/myapp
```

```
------------------------------------------------------
------
Q3. List out any five Kubernetes commands and give its
usage.
------------------------------------------------------
------
1. kubectl get pods
   - Lists all pods running in the cluster.
   - Example: kubectl get pods
2. kubectl apply -f file.yaml
   - Creates or updates resources from a YAML
configuration file.
   - Example: kubectl apply -f deployment.yaml
3. kubectl create deployment myapp --image=nginx
   - Creates a new deployment with the specified
image.
   - Example: kubectl create deployment myapp --
image=nginx
4. kubectl get deployments
   - Shows all deployments in the cluster.
   - Example: kubectl get deployments
5. kubectl delete deployment myapp
   - Deletes a deployment.
   - Example: kubectl delete deployment myapp
======================================================
==========
PROGRAM-WISE ANSWERS (Only the unique Q4/Q5 per
program)
======================================================
==========
------------------------------------------------------
------
PROGRAM 1 - Java: Docker + GitHub + DockerHub
------------------------------------------------------
------
Step 1: Write a simple Java program (Hello.java)
------------------------------------------------
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from Java!");
    }
}
Step 2: Create a Dockerfile
---------------------------
FROM openjdk:11
COPY Hello.java .
RUN javac Hello.java
CMD ["java", "Hello"]
Step 3: Build and run the Docker image
--------------------------------------
docker build -t java-hello .
docker run java-hello
Step 4: Push code to GitHub
---------------------------
git init
git add .
git commit -m "Java Hello World"
```

```
git remote add origin
https://github.com/yourusername/java-hello.git
git push origin main
Step 5: Push Docker image to DockerHub
---------------------------------------
docker login
docker tag java-hello yourusername/java-hello
docker push yourusername/java-hello
-------------------------------------------------------
------
PROGRAM 2 - C: Docker + GitHub + DockerHub
-------------------------------------------------------
------
Step 1: Write a simple C program (hello.c)
-----------------------------------------
#include <stdio.h>
int main() {
    printf("Hello from C!\n");
    return 0;
}
Step 2: Create a Dockerfile
---------------------------
FROM gcc:latest
COPY hello.c .
RUN gcc -o hello hello.c
CMD ["./hello"]
Step 3: Build and run the Docker image
--------------------------------------
docker build -t c-hello .
docker run c-hello
Step 4: Push code to GitHub
---------------------------
git init
git add .
git commit -m "C Hello World"
git remote add origin
https://github.com/yourusername/c-hello.git
git push origin main
Step 5: Push Docker image to DockerHub
--------------------------------------
docker login
docker tag c-hello yourusername/c-hello
docker push yourusername/c-hello
-------------------------------------------------------
------
PROGRAM 3 - HTML: Docker + GitHub + DockerHub
-------------------------------------------------------
------
Step 1: Write a simple HTML file (index.html)
---------------------------------------------
<!DOCTYPE html>
<html>
<body>
  <h1>Hello from HTML!</h1>
</body>
</html>
Step 2: Create a Dockerfile
---------------------------
```

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
Step 3: Build and run the Docker image
---------------------------------------
docker build -t html-hello .
docker run -p 8080:80 html-hello
(Open browser: http://localhost:8080)
Step 4: Push code to GitHub
----------------------------
git init
git add .
git commit -m "HTML Hello World"
git remote add origin
https://github.com/yourusername/html-hello.git
git push origin main
Step 5: Push Docker image to DockerHub
---------------------------------------
docker login
docker tag html-hello yourusername/html-hello
docker push yourusername/html-hello
------------------------------------------------------
------
PROGRAM 4 - Python: Docker + GitHub + DockerHub
------------------------------------------------------
------
Step 1: Write a simple Python program (app.py)
-----------------------------------------------
print("Hello from Python!")
Step 2: Create a Dockerfile
----------------------------
FROM python:3.9
COPY app.py .
CMD ["python", "app.py"]
Step 3: Build and run the Docker image
---------------------------------------
docker build -t python-hello .
docker run python-hello
Step 4: Push code to GitHub
----------------------------
git init
git add .
git commit -m "Python Hello World"
git remote add origin
https://github.com/yourusername/python-hello.git
git push origin main
Step 5: Push Docker image to DockerHub
---------------------------------------
docker login
docker tag python-hello yourusername/python-hello
docker push yourusername/python-hello
------------------------------------------------------
------
PROGRAM 5 - Node.js: Docker + GitHub + DockerHub
------------------------------------------------------
------
Step 1: Write a simple Node program (app.js)
---------------------------------------------
console.log("Hello from Node.js!");
```

```
Step 2: Create a Dockerfile
----------------------------
FROM node:18
COPY app.js .
CMD ["node", "app.js"]
Step 3: Build and run the Docker image
---------------------------------------
docker build -t node-hello .
docker run node-hello
Step 4: Push code to GitHub
----------------------------
git init
git add .
git commit -m "Node Hello World"
git remote add origin
https://github.com/yourusername/node-hello.git
git push origin main
Step 5: Push Docker image to DockerHub
---------------------------------------
docker login
docker tag node-hello yourusername/node-hello
docker push yourusername/node-hello
-----------------------------------------------------
------
PROGRAM 6 - Flask (Python): Docker + GitHub +
DockerHub
-----------------------------------------------------
------
Step 1: Write a Flask app (app.py)
-----------------------------------
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return "Hello from Flask!"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
Step 2: Create requirements.txt
--------------------------------
flask
Step 3: Create a Dockerfile
----------------------------
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
CMD ["python", "app.py"]
Step 4: Build and run the Docker image
---------------------------------------
docker build -t flask-app .
docker run -p 5000:5000 flask-app
(Open browser: http://localhost:5000)
Step 5: Push code to GitHub
----------------------------
git init
git add .
git commit -m "Flask Hello App"
```

```
git remote add origin
https://github.com/yourusername/flask-app.git
git push origin main
Step 6: Push Docker image to DockerHub
---------------------------------------
docker login
docker tag flask-app yourusername/flask-app
docker push yourusername/flask-app
-------------------------------------------------------
------
PROGRAM 7 - C Program + GitHub + Jenkins Job
-------------------------------------------------------
------
Step 1: Write a simple C program (hello.c)
-----------------------------------------
#include <stdio.h>
int main() {
    printf("Hello from C!\n");
    return 0;
}
Step 2: Push code to GitHub
----------------------------
git init
git add hello.c
git commit -m "C Program"
git remote add origin
https://github.com/yourusername/c-program.git
git push origin main
Step 3: Create a Jenkins Job
-----------------------------
1. Open Jenkins -> Click "New Item"
2. Enter job name -> Select "Freestyle Project" ->
Click OK
3. Under "Source Code Management" -> Select Git
4. Enter GitHub repo URL
5. Under "Build" -> Click "Add build step" -> Select
"Execute shell"
6. Enter the following commands:
   gcc hello.c -o hello
   ./hello
7. Click Save -> Click "Build Now"
8. Check Console Output to see the result.
-------------------------------------------------------
------
PROGRAM 8 - Python Calculator + GitHub + Jenkins Job
-------------------------------------------------------
------
Step 1: Write a Python calculator program (calc.py)
---------------------------------------------------
a = 10
b = 5
print("ADD:", a + b)
print("SUB:", a - b)
print("MUL:", a * b)
print("DIV:", a / b)
Step 2: Push code to GitHub
----------------------------
git init
```

```
git add calc.py
git commit -m "Python Calculator"
git remote add origin
https://github.com/yourusername/py-calc.git
git push origin main
Step 3: Create a Jenkins Job
-----------------------------
1. Open Jenkins -> Click "New Item"
2. Enter job name -> Select "Freestyle Project" ->
Click OK
3. Under "Source Code Management" -> Select Git
4. Enter GitHub repo URL
5. Under "Build" -> "Execute shell":
   python3 calc.py
6. Click Save -> Click "Build Now"
7. Check Console Output.
------------------------------------------------------
------
PROGRAM 9 - Java: Reverse of a Number + GitHub +
Jenkins
------------------------------------------------------
------
Step 1: Write Java program (Reverse.java)
-----------------------------------------
public class Reverse {
    public static void main(String[] args) {
        int num = 1234, rev = 0;
        while (num != 0) {
            rev = rev * 10 + num % 10;
            num /= 10;
        }
        System.out.println("Reversed: " + rev);
    }
}
Step 2: Push code to GitHub
-----------------------------
git init
git add Reverse.java
git commit -m "Java Reverse Number"
git remote add origin
https://github.com/yourusername/java-reverse.git
git push origin main
Step 3: Create a Jenkins Job
-----------------------------
1. Open Jenkins -> New Item -> Freestyle Project -> OK
2. Source Code Management -> Git -> Enter repo URL
3. Build -> Execute shell:
   javac Reverse.java
   java Reverse
4. Save -> Build Now -> Check Console Output.
------------------------------------------------------
------
PROGRAM 10 - Docker Hello-World + Jenkins Periodic
Trigger
------------------------------------------------------
------
Q4: Docker commands to run Hello-World image
---------------------------------------------
```

```
docker pull hello-world
docker run hello-world
Q5: Jenkins Job with Build Trigger (Periodic)
-----------------------------------------------
1. Open Jenkins -> New Item -> Freestyle Project -> OK
2. Under "Build Triggers" -> Check "Build
periodically"
3. Enter schedule: H/5 * * * *
   (This runs the job every 5 minutes)
4. Under "Build" -> Execute shell:
   echo "Jenkins build running!"
   date
5. Save -> The job will auto-trigger as per the
schedule.
-------------------------------------------------------
------
PROGRAM 11 - HTML Registration Form + GitHub + Jenkins
(Publish HTML)
-------------------------------------------------------
------
Step 1: Create HTML Registration Form (index.html)
-------------------------------------------------
<!DOCTYPE html>
<html>
<head><title>Registration Form</title></head>
<body>
  <h2>Registration Form</h2>
  <form>
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Password: <input type="password"
name="pass"><br><br>
    <input type="submit" value="Register">
  </form>
</body>
</html>
Step 2: Push code to GitHub
-----------------------------
git init
git add index.html
git commit -m "Registration Form"
git remote add origin
https://github.com/yourusername/html-form.git
git push origin main
Step 3: Jenkins Job to Publish HTML
-------------------------------------
1. Install "HTML Publisher" plugin in Jenkins
   (Manage Jenkins -> Plugins -> Search "HTML
Publisher" -> Install)
2. New Item -> Freestyle Project -> OK
3. Source Code Management -> Git -> Enter repo URL
4. Build -> Add Post-build Action -> "Publish HTML
reports"
5. HTML directory to archive: . (current folder)
   Index page: index.html
   Report title: Registration Form
6. Save -> Build Now -> View the HTML report from
Jenkins dashboard.
```

```
------------------------------------------------------
------
PROGRAM 12 - Jenkins Pipeline using Jenkinsfile (SCM -
Git)
------------------------------------------------------
------
Step 1: Create a Jenkinsfile in your repo
----------------------------------------
pipeline {
    agent any
    stages {
        stage('Clone') {
            steps {
                echo 'Cloning repository...'
            }
        }
        stage('Build') {
            steps {
                echo 'Building the project...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying...'
            }
        }
    }
}
Step 2: Push Jenkinsfile to GitHub
----------------------------------
git add Jenkinsfile
git commit -m "Added Jenkinsfile"
git push origin main
Step 3: Create Jenkins Pipeline Job (SCM)
----------------------------------------
1. New Item -> Pipeline -> OK
2. Under "Pipeline" section:
   - Definition: "Pipeline script from SCM"
   - SCM: Git
   - Repository URL: your GitHub repo URL
   - Script Path: Jenkinsfile
3. Save -> Build Now
4. Each stage will show as a separate block in the
pipeline view.
------------------------------------------------------
------
PROGRAM 13 - Java & Python Pattern + Jenkins (File
Parameterization)
------------------------------------------------------
------
Step 1: Java Pattern Program (Pattern.java)
-------------------------------------------
public class Pattern {
```

```
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++)
                System.out.print("* ");
            System.out.println();
        }
    }
}
Step 2: Python Pattern Program (pattern.py)
-------------------------------------------
for i in range(1, 6):
    print("* " * i)
Step 3: Create Jenkins Job with File Parameterization
-----------------------------------------------------
-
1. New Item -> Freestyle Project -> OK
2. Check "This project is parameterized"
3. Add Parameter -> "File Parameter"
   - File location: script_file
4. Under Build -> Execute shell:
   python3 ${script_file}   # or java Pattern
5. Save -> Build with Parameters -> Upload the script
file -> Build
-----------------------------------------------------
------
PROGRAM 14 - Ubuntu Docker Container + Jenkins 5-Stage
Pipeline
-----------------------------------------------------
------
Q4: Docker commands for Ubuntu container named
"MyContainer"
-----------------------------------------------------
--------
docker pull ubuntu
docker run -it --name MyContainer ubuntu /bin/bash
# Inside the container, run shell commands:
ls
pwd
echo "Hello from MyContainer"
cat /etc/os-release
exit
Q5: Jenkins Pipeline with 5 Stages
-----------------------------------
pipeline {
    agent any
    stages {
        stage('Stage 1: Clone') {
            steps { echo 'Cloning code from GitHub' }
        }
        stage('Stage 2: Install') {
            steps { echo 'Installing dependencies' }
        }
        stage('Stage 3: Build') {
            steps { echo 'Building the application' }
        }
        stage('Stage 4: Test') {
            steps { echo 'Running unit tests' }
        }
```

```
        stage('Stage 5: Deploy') {
            steps { echo 'Deploying the application' }
        }
    }
}
```

Steps to create in Jenkins:

1. New Item -> Pipeline -> OK
2. Pipeline section -> Pipeline script
3. Paste the above pipeline script
4. Save -> Build Now

---

## PROGRAM 15 - Python Docker Image + Kubernetes nginx Deployment

---

### Q4: Docker commands to run Python image

```
docker pull python:3.9
docker run -it python:3.9 /bin/bash
# Inside container:
python3 -c "print('Hello from Python container')"
python3 -c "print(2 + 3)"
exit
```

### Q5: Kubernetes - Create nginx deployment and run kubectl commands

```
# Create deployment
kubectl create deployment nginx-deploy --image=nginx
# Check deployment
kubectl get deployments
# Check pods
kubectl get pods
# Expose the deployment
kubectl expose deployment nginx-deploy --port=80 --
type=NodePort
# Check services
kubectl get services
# Delete deployment
kubectl delete deployment nginx-deploy
```

---

## PROGRAM 16 - Node Docker Image + Kubernetes Python Deployment

---

### Q4: Docker commands to run Node image

```
docker pull node:18
docker run -it node:18 /bin/bash
# Inside container:
node -e "console.log('Hello from Node container')"
node -e "console.log(5 * 10)"
exit
```

### Q5: Kubernetes - Create Python deployment and run kubectl commands

```
-------------------------------------------------------
------------
# Create deployment
kubectl create deployment python-deploy --
image=python:3.9
# Check pods
kubectl get pods
# Describe deployment
kubectl describe deployment python-deploy
# Scale the deployment
kubectl scale deployment python-deploy --replicas=2
# Check pods again
kubectl get pods
# Delete deployment
kubectl delete deployment python-deploy
-------------------------------------------------------
------
PROGRAM 17 - Nginx Docker Image + Kubernetes MongoDB
Deployment
-------------------------------------------------------
------
Q4: Docker commands to run nginx image
----------------------------------------
docker pull nginx
docker run -d -p 8080:80 --name my-nginx nginx
# Check running container
docker ps
# Execute commands inside container
docker exec -it my-nginx /bin/bash
ls /usr/share/nginx/html
exit
# Stop and remove
docker stop my-nginx
docker rm my-nginx
Q5: Kubernetes - Create MongoDB deployment and run
kubectl commands
-------------------------------------------------------
-------------
# Create deployment
kubectl create deployment mongo-deploy --image=mongo
# Check deployment
kubectl get deployments
# Check pods
kubectl get pods
# Describe a pod
kubectl describe pod <pod-name>
# Delete deployment
kubectl delete deployment mongo-deploy
-------------------------------------------------------
------
PROGRAM 18 - HTML Registration Form: Docker +
DockerHub + GitHub
-------------------------------------------------------
------
Step 1: Create HTML Registration Form (index.html)
---------------------------------------------------
<!DOCTYPE html>
<html>
```

```
<head><title>Registration</title></head>
<body>
  <h2>Registration Form</h2>
  <form>
    Name: <input type="text"><br><br>
    Email: <input type="email"><br><br>
    Phone: <input type="tel"><br><br>
    <input type="submit" value="Register">
  </form>
</body>
</html>
Step 2: Create Dockerfile
-------------------------
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
Step 3: Build and run
---------------------
docker build -t html-reg-app .
docker run -p 8080:80 html-reg-app
Step 4: Push code to GitHub
---------------------------
git init
git add .
git commit -m "HTML Registration App"
git remote add origin
https://github.com/yourusername/html-reg.git
git push origin main
Step 5: Push image to DockerHub
-------------------------------
docker login
docker tag html-reg-app yourusername/html-reg-app
docker push yourusername/html-reg-app
-----------------------------------------------------
------
PROGRAM 19 - Docker Compose: Two BusyBox containers
(ping test)
-----------------------------------------------------
------
Step 1: Create docker-compose.yml
---------------------------------
version: '3'
services:
  bbConA:
    image: busybox
    container_name: bbConA
    command: sh -c "ping -c 4 bbConB"
    depends_on:
      - bbConB
    networks:
      - mynet
  bbConB:
    image: busybox
    container_name: bbConB
    command: sh -c "sleep 30"
    networks:
      - mynet
networks:
  mynet:
```

```
    driver: bridge
Step 2: Run the Docker Compose file
-----------------------------------
docker-compose up
(You will see bbConA pinging bbConB successfully in
the output)
Step 3: Stop and remove
-----------------------
docker-compose down
-------------------------------------------------------
------
PROGRAM 20 - Kubernetes: nginx Deployment using YAML
-------------------------------------------------------
------
Step 1: Create deployment YAML file (nginx-
deployment.yaml)
-------------------------------------------------------
-------
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
Step 2: Apply the YAML file
---------------------------
kubectl apply -f nginx-deployment.yaml
Step 3: Check the deployment
----------------------------
kubectl get deployments
kubectl get pods
Step 4: Delete deployment
-------------------------
kubectl delete -f nginx-deployment.yaml
-------------------------------------------------------
------
PROGRAM 21 - Kubernetes Node Deployment + Selenium
(Open Website)
-------------------------------------------------------
------
Q4: Kubernetes - Create Node deployment and run
kubectl commands
-------------------------------------------------------
----------
kubectl create deployment node-deploy --image=node:18
```

```
kubectl get deployments
kubectl get pods
kubectl describe deployment node-deploy
kubectl delete deployment node-deploy
Q5: JavaScript program to open Google using Selenium
-----------------------------------------------------
Step 1: Install Selenium and ChromeDriver
-----------------------------------------
npm install selenium-webdriver
Step 2: Write the JavaScript program (openGoogle.js)
-----------------------------------------------------
const { Builder } = require('selenium-webdriver');
async function openGoogle() {
    let driver = await new
Builder().forBrowser('chrome').build();
    await driver.get('https://www.google.com');
    console.log("Google opened successfully!");
    await driver.quit();
}
openGoogle();
Step 3: Run the program
-----------------------
node openGoogle.js
-----------------------------------------------------
------
PROGRAM 22 - Selenium: Login Validation
-----------------------------------------------------
------
Step 1: Write a simple HTML login page (login.html)
---------------------------------------------------
<!DOCTYPE html>
<html>
<body>
  <h2>Login</h2>
  <input type="text" id="username"
placeholder="Username"><br><br>
  <input type="password" id="password"
placeholder="Password"><br><br>
  <button onclick="validate()">Login</button>
  <p id="msg"></p>
  <script>
    function validate() {
      var u =
document.getElementById("username").value;
      var p =
document.getElementById("password").value;
      if (u === "admin" && p === "1234") {
        document.getElementById("msg").innerText =
"Login Successful!";
      } else {
        document.getElementById("msg").innerText =
"Invalid Credentials!";
      }
    }
  </script>
</body>
</html>
Step 2: Selenium test (loginTest.js)
```

```
-------------------------------------
const { Builder, By } = require('selenium-webdriver');
const path = require('path');
async function testLogin() {
    let driver = await new
Builder().forBrowser('chrome').build();
    await driver.get('file://' +
path.resolve('login.html'));
    await
driver.findElement(By.id('username')).sendKeys('admin'
);
    await
driver.findElement(By.id('password')).sendKeys('1234')
;
    await
driver.findElement(By.css('button')).click();
    let msg = await
driver.findElement(By.id('msg')).getText();
    console.log("Result:", msg);
    await driver.quit();
}
testLogin();
Step 3: Run
------------
node loginTest.js
-----------------------------------------------------
------
PROGRAM 23 - Selenium: Check Exam Results on College
Website
-----------------------------------------------------
------
Step 1: Selenium program (checkResult.js)
-----------------------------------------
const { Builder, By, until } = require('selenium-
webdriver');
async function checkResult() {
    let driver = await new
Builder().forBrowser('chrome').build();
    await driver.get('https://matrusri.skolo.in/');
    console.log("Opened College Website");
    // Wait for page to load
    await driver.wait(until.titleContains(''), 5000);
    let title = await driver.getTitle();
    console.log("Page Title:", title);
    await driver.quit();
}
checkResult();
Step 2: Run
------------
node checkResult.js
-----------------------------------------------------
------
PROGRAM 24 - Selenium: Calculator Web App with Test
Cases
-----------------------------------------------------
------
Step 1: Create Calculator HTML (calculator.html)
-------------------------------------------------
```

```
<!DOCTYPE html>
<html>
<body>
  <h2>Calculator</h2>
  <input type="number" id="a" placeholder="Number 1">
  <input type="number" id="b" placeholder="Number 2">
  <button onclick="calc('add')">Add</button>
  <button onclick="calc('sub')">Sub</button>
  <p id="result"></p>
  <script>
    function calc(op) {
      var a =
parseFloat(document.getElementById('a').value);
      var b =
parseFloat(document.getElementById('b').value);
      var res;
      if (op === 'add') res = a + b;
      else if (op === 'sub') res = a - b;
      document.getElementById('result').innerText =
"Result: " + res;
    }
  </script>
</body>
</html>
Step 2: Selenium Test (calcTest.js)
-------------------------------------
const { Builder, By } = require('selenium-webdriver');
const path = require('path');
async function testCalc() {
    let driver = await new
Builder().forBrowser('chrome').build();
    await driver.get('file://' +
path.resolve('calculator.html'));
    await
driver.findElement(By.id('a')).sendKeys('10');
    await
driver.findElement(By.id('b')).sendKeys('5');
    await
driver.findElement(By.css('button')).click();
    let result = await
driver.findElement(By.id('result')).getText();
    console.log("Test Result:", result);
    await driver.quit();
}
testCalc();
Step 3: Run
------------
node calcTest.js
------------------------------------------------------
------
PROGRAM 25 - Jenkins: Java Program in GitHub + Remote
Trigger
------------------------------------------------------
------
Step 1: Write a Java program (Hello.java) and push to
GitHub
(Same as Program 1 - see above)
Step 2: Create Jenkins Job
```

```
--------------------------
1. New Item -> Freestyle Project -> OK
2. Source Code Management -> Git -> Enter repo URL
3. Under "Build Triggers" -> Check "Trigger builds
remotely"
4. Enter a token name, e.g.: mytoken
5. Build -> Execute shell:
   javac Hello.java
   java Hello
6. Save
Step 3: Trigger the build remotely
-----------------------------------
Open browser and go to:
http://localhost:8080/job/JobName/build?token=mytoken
This will trigger the build remotely without logging
into Jenkins.
------------------------------------------------------
------
PROGRAM 26 - Jenkins: Python Calculator + SCM Polling
Trigger
------------------------------------------------------
------
Step 1: Write Python Calculator (calc.py)
-----------------------------------------
a = 10
b = 5
print("ADD:", a + b)
print("SUB:", a - b)
print("MUL:", a * b)
print("DIV:", a / b)
Step 2: Push to GitHub
----------------------
git init
git add calc.py
git commit -m "Python Calculator"
git remote add origin
https://github.com/yourusername/py-calc.git
git push origin main
Step 3: Create Jenkins Job with SCM Polling
-------------------------------------------
1. New Item -> Freestyle Project -> OK
2. Source Code Management -> Git -> Enter repo URL
3. Under "Build Triggers" -> Check "Poll SCM"
4. Enter schedule: H/5 * * * *
   (Checks GitHub every 5 minutes for new changes)
5. Build -> Execute shell:
   python3 calc.py
6. Save
Now whenever you push a new change to GitHub, Jenkins
will
automatically detect it and trigger a new build!
------------------------------------------------------
------
PROGRAM 27 - Jenkins: Java & Python Calculator + File
and Variable Parameterization
------------------------------------------------------
------
Step 1: Java Calculator (Calculator.java)
```

```
----------------------------------------
public class Calculator {
    public static void main(String[] args) {
        int a = 10, b = 5;
        System.out.println("ADD: " + (a + b));
        System.out.println("SUB: " + (a - b));
        System.out.println("MUL: " + (a * b));
        System.out.println("DIV: " + (a / b));
    }
}
Step 2: Python Calculator (calc.py)
-----------------------------------
a = 10
b = 5
print("ADD:", a + b)
print("SUB:", a - b)
print("MUL:", a * b)
print("DIV:", a / b)
Step 3: Jenkins Job with File + Variable
Parameterization
-------------------------------------------------------
----
1. New Item -> Freestyle Project -> OK
2. Check "This project is parameterized"
3. Add Parameter -> "String Parameter"
   - Name: LANG
   - Default Value: python
   - Description: Enter 'java' or 'python'
4. Add Parameter -> "File Parameter"
   - File location: uploaded_script
5. Build -> Execute shell:
   if [ "$LANG" = "python" ]; then
       python3 calc.py
   else
       javac Calculator.java
       java Calculator
   fi
6. Save -> Build with Parameters
   - Enter LANG = python or java
   - Upload the corresponding script file
   - Click Build
=======================================================
==========
END OF DEVOPS LAB ANSWERS
=======================================================
==========
```